MINISTRY OF EDUCATION
IMAM ABDULRAHMAN BIN
FAISAL UNIVERSITY
COLLEGE OF COMPUTER SCIENCE
& INFORMATION TECHNOLOGY
Department of Computer Science
وزارة التعليم
جامعة الإمام
عبدالرحمن بن فيصل
كلية علوم الحاسب
وتقنية المعلومات
جامعة الإمام عبد الرحمن بن فيصل
IMAM ABDULRAHMAN BIN FAISAL UNIVERSITY

ASSIGNMENT 1

Project Proposal and Financial Support Application
ARTI 503 Parallel Architecture and Programming

Term 1 2026-2027

Course title	Parallel Architecture and Programming
Course code	ARTI 503
Instructor	Dr. Rabab Alkhalifa

Submission Deadline: 4 October 2026

Project Topic: Parallelizing a Compute-Intensive Image Processing Pipeline: Speed-Up, Efficiency and Scalability Analysis of Sequential and Parallel Implementations

Group No. 2

No.	*Student Name*	*Student ID*	*Role*
1	Moudi Alameel	2230004961	Leader
2	Fatimah Bukhamseen	2230002914	Member
3	Wilayah Al Wuhayb	2230005246	Member
4	Fatima Alzayer	2230006541	Member
5	Jumanah Alhareth	2230004709	Member
6	Zahra Alqassab	2230002986	Member
7	Ghada Alsubaie	2230005340	Member
8	Noor Alkhames	2230003700	Member
9	Zainab Gasgous	2230005218	Member
10	Saja Nabeel Alghasham	2230003764	Member

Introduction:

Image processing is used in many fields, including medical imaging, satellite and drone imagery, surveillance, and quality inspection in manufacturing. These applications handle very large collections of high-resolution images, and each image needs millions of pixel operations. Operations such as filtering, edge detection and contrast enhancement apply the same computation to every pixel, so they are a natural fit for parallel computing. Modern multi-core CPUs and GPUs can run these operations on many pixels or many images at once. This project studies how much performance can be gained by moving a compute-intensive image-processing pipeline from sequential to parallel execution, and what limits that gain.

Problem Statement:

Sequential image processing does not scale. The time to process a batch of images grows with both the image resolution and the number of images, and neighborhood operations such as Gaussian blur make this worse because each pixel depends on many others. As a result, large image workloads take a long time on a single processor core, which delays analysis and wastes available hardware. The problem is to find out how effectively an image-processing pipeline (grayscale conversion, Gaussian blur, Sobel edge detection and histogram equalisation) can be parallelized. We need to measure the speed-up achieved, find the point where overheads such as synchronization and communication limit the gains, and understand the cost of the resources needed to reach those gains.

Detailed Description:

This project compares sequential and parallel implementations of an image-processing pipeline (grayscale conversion, Gaussian blur, Sobel edge detection and histogram equalisation). We will parallelize across images and within images, then measure execution time, speed-up, efficiency, scalability and cost across different image sizes and worker counts. Cloud computing resources are requested to run the experiments at scale.

Suggested Solution:

We will follow four stages:

1. **Sequential Baseline:** Implement the full pipeline as a single-threaded program and use it as the reference for correctness and for timing (T1). The baseline processes a set of images at several resolutions.
2. **Parallel Implementation:**
    - **Across Images**: each image is an independent task, distributed to a pool of worker processes.
    - **Within an Image**: each image is divided into blocks of rows, and workers process the blocks at the same time. Overlapping border rows (halos) are included so that the blur and Sobel filters give correct results.

- **Histogram Equalisation***:* each worker computes a partial histogram, and the partial results are then combined. This needs synchronization, and we will measure how much it costs.

3. **Experiments:** run both versions on cloud instances while varying image size, number of images and number of workers, and repeat each run to reduce noise.
4. **Evaluation:** compare the parallel output to the sequential output to confirm correctness, and calculate execution time, speed-up (T1/Tp), efficiency (speed-up divided by workers), scalability and cost. Then analyze the results to explain where parallelization helps and where overhead limits it.

Budget Justification:

AWS Cloud resources are needed because the experiments must be run at several scales and worker counts, including more cores than a student laptop has. We requested for a total of 1000$ of resources. Refer to the table below for details on the requested items.

Table 1 - Requested Items and Total Budget

Item	Purpose	Estimated usage	Estimated cost (USD)
Multi-core CPU instance (e.g. 16 vCPU)	Scalability tests with 1–16 workers	~200 hours	~140
GPU instance (e.g. 1 entry-level GPU)	GPU test of blur and Sobel	~100 hours	~55
Storage (image datasets and results)	Store input images and output files	~100 GB for the project period	~20
Data transfer and misc.	Download results and datasets	–	~10
Contingency (~25%)	Repeated runs and re-testing	–	~60
AWS SageMaker	Includes all services related to GPUs	-	~606
Total requested			**≈ 996**

Timeline:

Table 2 - Timeline Table

Week	Activities	Deliverable
1	Form the group, confirm the topic, review related work, identify the funding source and submit the application	Application submitted

2	Review the funder's feedback and peer evaluation, then revise the plan if needed. Prepare the image dataset at several resolutions	Dataset and refined project plan
3	Implement the sequential pipeline (grayscale conversion, Gaussian blur, Sobel edge detection, histogram equalisation)	Working sequential version
4	Verify the sequential output, profile the code to find the slowest steps, and record baseline times (T1)	Baseline timing report
5	Set up the cloud environment, check access and test the code on a cloud instance	Working cloud setup
6	Implement parallelism across images (a pool of worker processes)	Parallel version 1
7	Implement parallelism within images (row blocks with halo rows for blur and Sobel)	Parallel version 2
8	Implement parallel histogram equalisation (partial histograms combined) and check that outputs match the sequential version	Correctness-verified parallel pipeline
9	Optional GPU implementation and testing of blur and Sobel, if resources allow	GPU version (optional)
10	Run experiments on the cloud, varying image size, number of images and number of workers, with repeated runs	Raw benchmark data
11	Calculate execution time, speed-up, efficiency, scalability and cost. Produce graphs and tables	Results and graphs
12	Analyze the results, including where synchronization and overhead limit the gains, and write the final report	Draft report
13	Revise the report, finalize the code and submit all files	Final report and code

AWS Financial Support Application:

- **Description:**

We are applying for the AWS Cloud Credit for Research program to secure $1,000 in funding for our cloud infrastructure needs, specifically GPUs. CPUs are free to access in AWS.

- **Work Completed:**

We have finalized the core project proposal, detailing our problem statement, methodology, and the specific AWS architecture required. Additionally, we generated an AWS Pricing Calculator estimate to justify the $1,000 funding request over the course of our project timeline.

- **Reason for Selecting It:**

We are familiar with working with AWS and have already provisioned an EC2 instance to test on CPUs.

- **Evidence of Submitting the Application:**

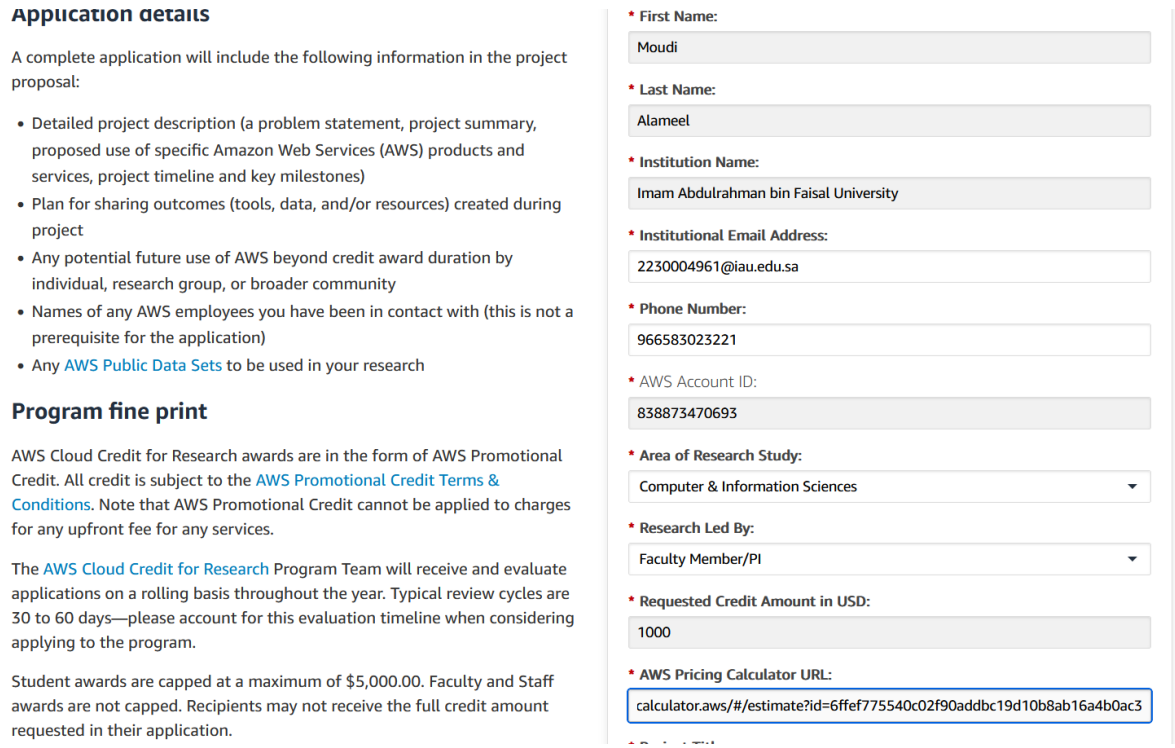


Figure 1 - AWS Application Form

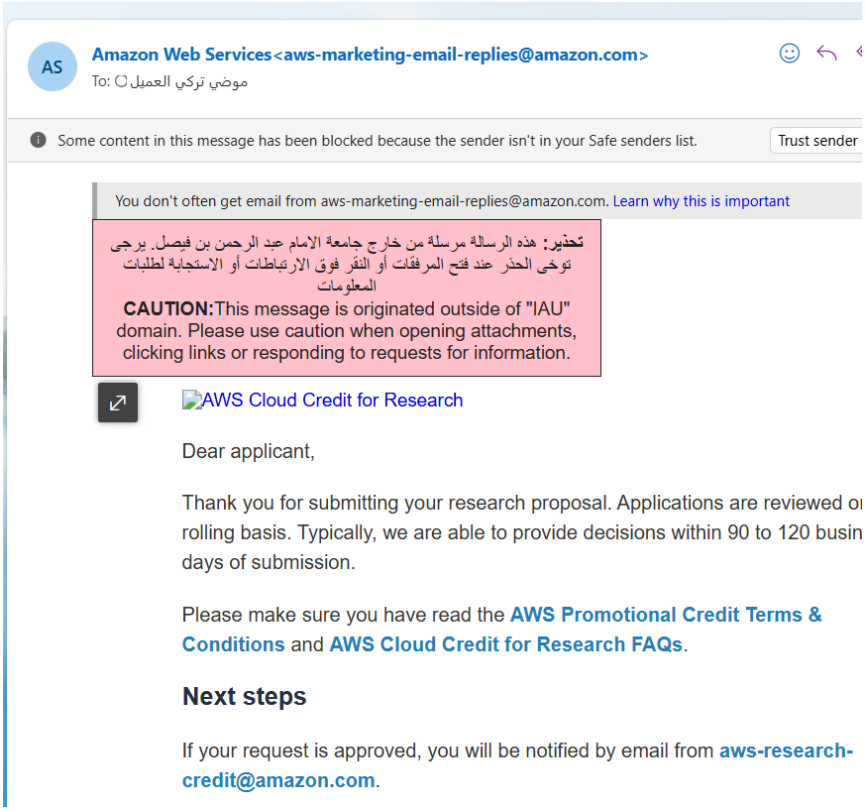


Figure 2 - Submission Evidence